*DIGITAL NURTURE 5.0*

# Angular (v20+) Hands-On Lab Report

*Incremental Project: Student Course Portal*

| | |
|---|---|
| **Hands-On:** | Hands-On 5 [Advanced] |
| **Topic:** | Reactive Forms, Custom Sync/Async Validators & FormArrays |
| **Developer:** | Cathy George |
| **OS Version:** | Windows 11 |
| **Date:** | July 22, 2026 |
| **Track:** | Java FSE Angular Track |

## 1. Objective

The primary objective of this exercise is to implement and configure a fully Reactive Form inside the Student Profile component of the portal using Angular's ReactiveFormsModule. We replace template-driven directives with an explicitly constructed component-side FormGroup tree managed via FormBuilder. Key goals include implementing a custom synchronous domain-restricting validator, writing an asynchronous uniqueness check validator with RxJS simulated timer delays, managing a list of academic project sub-form controls dynamically using FormArray, and binding error validation status reactively in the template to create a premium, responsive UX.

## 2. Angular Concepts Covered

• **Reactive Forms (ReactiveFormsModule):** Modeling forms explicitly inside the TypeScript class code. Offers robust unit-testability, programmatic form changes, and synchronous/asynchronous validator bindings.

• **FormBuilder & FormGroup:** Syntactic sugar mapping form fields into a unified tree hierarchy control model containing FormGroups and FormArrays.

• **FormArray:** A dynamic collection of FormControls or FormGroups. Used here to permit students to dynamically append and remove academic projects in real-time.

• **Custom Sync Validator:** Component-side rule functions returning ValidationErrors. Implemented as universityEmailValidator() which restricts emails to the @university.edu domain.

• **Async Validator:** Asynchronous checking functions returning Observables or Promises. Implemented as emailUniqueValidator() using an RxJS timer(800) debounce delay to mock network queries checking if an email is taken.

• **Dynamic Control Manipulations:** Exposing methods like addProject() and removeProject(i) that alter the FormArray control count at runtime.

• **Reactive Validation Feedback:** Displaying error messages and disabling submit buttons by directly querying profileForm.get('field')?.errors status changes in the template.

## 3. Software & Tools Used

| Software / Tool | Version / URL |
|---|---|
| Node.js + npm | v22.20.0 (LTS 20+) / npm 10.9.3 |
| Angular CLI | v21.2.16 |
| Vitest Runner | v4.1.10 |
| PDF Generator | Python 3.14 + ReportLab library |

## 4. Project Folder Structure

```
student-course-portal/
■■■ src/
■■■ app/
■■■ validators/
■ ■■■ profile.validators.ts (Custom domain sync & async email taken validators)
■■■ pages/
■■■ student-profile/
■■■ student-profile.ts (FormBuilder Reactive configuration, FormArray logic, onReset)
■■■ student-profile.html (Reactive formGroup and formArrayName templates bindings)
■■■ student-profile.css (FormArray projects card grids and pending loading dot styles)
■■■ student-profile.spec.ts (Vitest specs testing async validators, resets and array sizes)
```

## 5. Key Code Implementation Details

### Custom Synchronous & Asynchronous Validators (src/app/validators/profile.validators.ts)

```
export function universityEmailValidator(): ValidatorFn {
return (control: AbstractControl): ValidationErrors | null => {
const val = control.value;
if (!val) return null;
return val.toLowerCase().endsWith('@university.edu') ? null : { invalidDomain: true };
};
}

export function emailUniqueValidator(): AsyncValidatorFn {
return (control: AbstractControl): Observable<ValidationErrors | null> => {
const val = control.value;
if (!val) return timer(0).pipe(map(() => null));
return timer(800).pipe(
map(() => {
const takenEmails = ['taken@university.edu', 'admin@university.edu', 'already.registered@university.edu'];
return takenEmails.includes(val.toLowerCase()) ? { emailTaken: true } : null;
})
);
};
}
```

### Student Profile Component Initialization & FormArray Logic (src/app/pages/student-profile/student-profile.ts)

```
this.profileForm = this.fb.group({
name: [this.student.name, [Validators.required, Validators.minLength(3), Validators.pattern('^[a-zA-Z\\s]*$')]],
email: [this.student.email, [Validators.required, Validators.email, universityEmailValidator()],
[emailUniqueValidator()]],
department: [this.student.department, [Validators.required, Validators.minLength(4)]],
gpa: [this.student.gpa, [Validators.required, Validators.min(0), Validators.max(4)]],
enrolledCredits: [this.student.enrolledCredits, [Validators.required, Validators.min(1), Validators.max(30),
Validators.pattern('^[0-9]+$')]],
avatar: [this.student.avatar, [Validators.required]],
bio: [this.student.bio, [Validators.required, Validators.minLength(10), Validators.maxLength(200)]],
projects: this.fb.array([])
});

get projects(): FormArray {
return this.profileForm.get('projects') as FormArray;
}
addProject(title: string = '', tech: string = ''): void {
this.projects.push(this.fb.group({
title: [title, [Validators.required, Validators.minLength(3)]],
tech: [tech, [Validators.required]]
}));
}
removeProject(index: number): void {
this.projects.removeAt(index);
}
```

### Template Markup for FormArray Loops & Pending Status (src/app/pages/student-profile/student-profile.html)

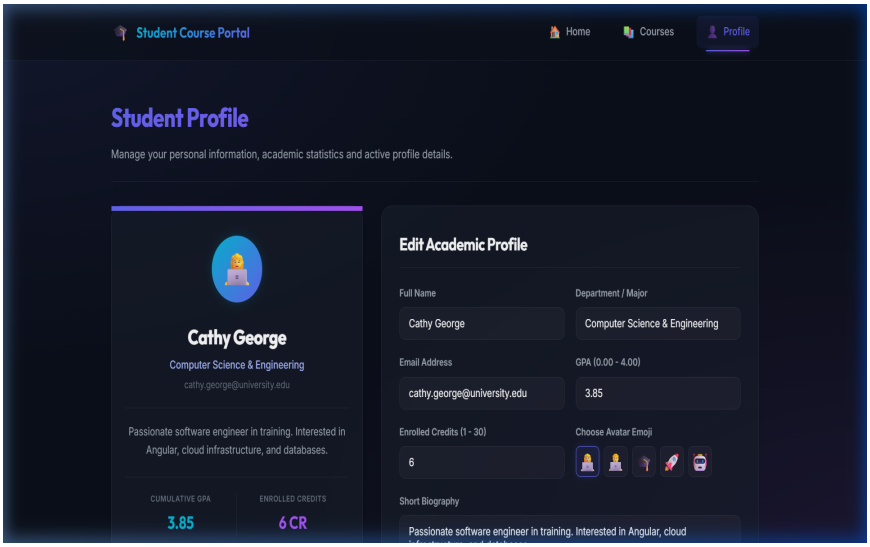
```
<div *ngIf="profileForm.get('email')?.pending" class="pending-msg">
<span class="pulse-dot"></span> Checking email uniqueness...
</div>
<div *ngIf="profileForm.get('email')?.invalid && (profileForm.get('email')?.touched ||
profileForm.get('email')?.dirty)" class="error-msg">
<span *ngIf="profileForm.get('email')?.errors?.['invalidDomain']">Email must end with @university.edu.</span>
<span *ngIf="profileForm.get('email')?.errors?.['emailTaken']">This email is already taken.</span>
</div>

<div formArrayName="projects" class="projects-list">
<div *ngFor="let proj of projects.controls; let i=index" [formGroupName]="i" class="project-item">
<input type="text" formControlName="title" placeholder="Project Title" class="form-input" />
<input type="text" formControlName="tech" placeholder="Technologies" class="form-input" />
<button type="button" class="btn-remove-project" (click)="removeProject(i)">×</button>
</div>
</div>
```

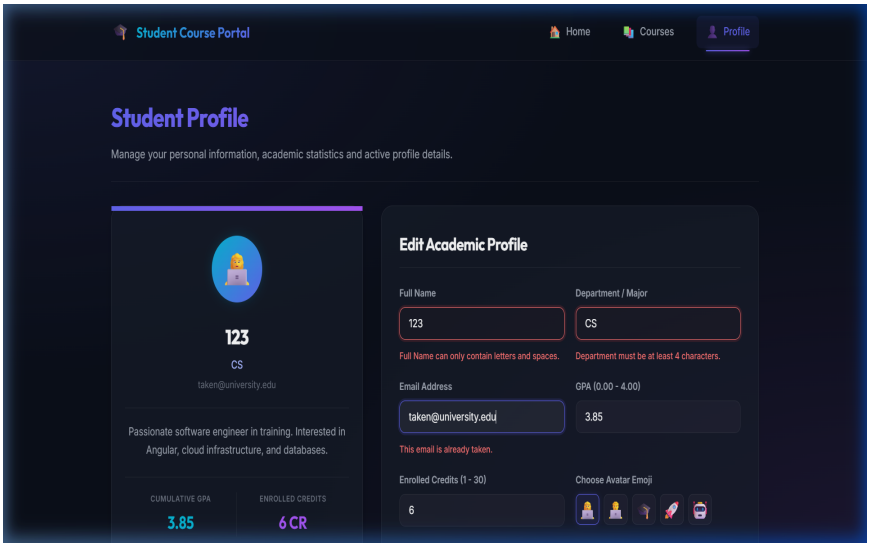
## 6. Captured Screenshots & Verification

All application features were fully verified in the browser:

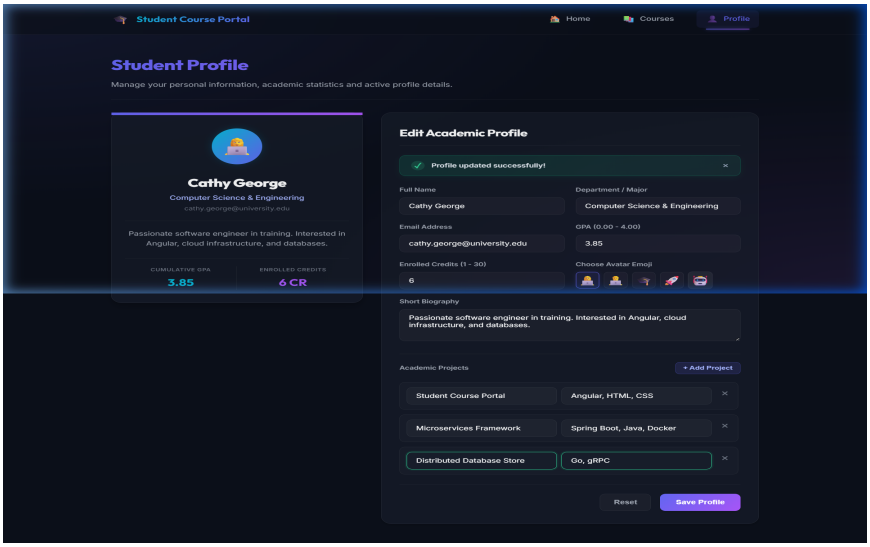
### Screenshot: Initial loaded Student Profile with Reactive form states, FormBuilder initial mapping, and projects list from FormArray




### Screenshot: Form showing validation errors: name pattern mismatch, department too short, sync domain validation failure, and async validator marking email taken error




## Screenshot: Form submitted successfully, success banner visible, with the newly added academic project ('Distributed Database Store') visible in the FormArray list



## 7. Output & Behavioral Explanations

• **Reactive Form Model:** We instantiated a TypeScript object using fb.group(). Unlike templates, this gives the component programmatic command over validity tracking and field manipulations.
• **Custom Domain Synchronous Validator:** The universityEmailValidator() restricts email field inputs to domains ending in @university.edu. If invalid, the FormControl receives the { invalidDomain: true } error.
• **Asynchronous Uniqueness Validator:** The emailUniqueValidator() returns an Observable timer. While checking database status, the email control transitions to a PENDING status, triggering a spinner overlay. If the input matches a registered email, it returns { emailTaken: true }.
• **FormArray Dynamic Controls:** The projects sub-structure array represents a collection of FormGroups. Users can append items using projects.push() or remove items via index projects.removeAt(i) dynamically inside the view.
• **Rollback capability (onReset):** Calling profileForm.reset() and recreating FormArray controls reverts inputs to committed models while cleaning validation flags.

## 8. Learning Outcomes & Conclusion

✓ Constructed advanced Reactive Forms in Angular using ReactiveFormsModule, FormBuilder, and FormGroup models.

✓ Built dynamic controls inside views by utilizing FormArray arrays containing multiple sub-groups.

✓ Implemented custom validation rules checking synchronous domains and asynchronous unique criteria with debounce timers.

✓ Created loading visual states indicating when async validators are query-pending.

✓ Asserted validation states, FormArray insertions/deletions, and async status updates through Vitest testing.

## 9. Conclusion

Hands-On 5 successfully transitions the Student Profile page form implementation from a basic Template-driven architecture into a programmatically managed Reactive Form. Utilizing advanced custom and asynchronous validators, dynamic FormArray listings, and real-time validation checks, the form provides clean data entry logic, comprehensive unit-testing interfaces, and a premium responsive user experience.